

PART 2 OF 2 — THE HOW

The HOW — Code & Pipelines

A technical deep dive: run the model, rebuild the data, train teacher+student, evaluate honestly, and build it from scratch.

Companion to Part 1 (The WHY)

Every code block is runnable or a faithful skeleton

Official code = from the model card/paper · **Reconstructed** = reproduces described method

How to trust the code in this deck

OFFICIAL

Comes straight from the Hugging Face model card or the paper's stated config. Marked **"official"**.

RECONSTRUCTED

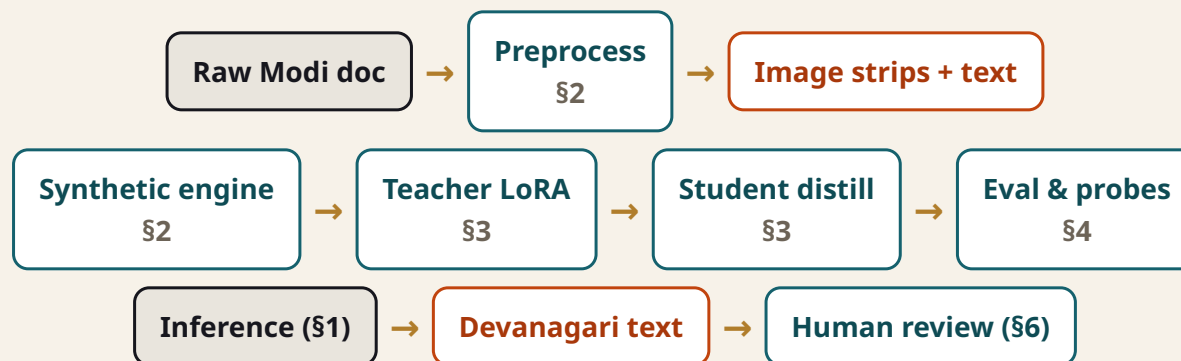
Reproduces a method the paper *describes* but doesn't ship. Marked **"reconstructed"** — correct in spirit, tune before production.

ILLUSTRATIVE

How you'd measure something the paper *didn't* (CER on damage, Brahmi zero-shot, privacy). Marked **"illustrative"**.

! This honesty matters: the released artifact is a Gemma **teacher** LoRA; the distilled student and full training code are not public. We rebuild them from the paper.

One mental model for the entire deck



i We start at the end (§1 inference — the fastest win), then go back and rebuild each upstream stage.

Environment — install once

setup.sh – official deps (versions are the practical floor)

```
# Python 3.10+, a CUDA GPU with >=24GB VRAM for the 12B teacher in bf16
pip install "torch>=2.2" torchvision --index-url https://download.pytorch.org/whl/cu121
pip install "transformers>=4.50" peft accelerate pillow
pip install datasets sacrebleu jiwer          # data + metrics
pip install opencv-python-headless numpy      # preprocessing
pip install huggingface_hub                   # gated-repo login

# the model is GATED: accept terms on the model page, then:
huggingface-cli login                        # paste a token with read access
```

! No GPU? You can still run §2 (data) and study §3/§4 code. Inference of a 12B model on CPU is impractically slow.

01

Run the released model

Load Gemma + the LoRA adapter and transliterate an image.

Step 1 — load base + adapter (official)

`infer_load.py` – official (the adapter only works on this exact base)

```
import torch
from transformers import AutoModelForImageTextToText, AutoProcessor
from peft import PeftModel

BASE      = "google/gemma-3-12b-it"
ADAPTER   = "historyHulk/ModiTrans-12B-Gemma-Teacher"
device    = "cuda:0"

# 1) load the base Vision-Language Model
model = AutoModelForImageTextToText.from_pretrained(
    BASE, torch_dtype=torch.bfloat16, device_map=device)

# 2) attach the trained LoRA adapter on top (this IS the Modi skill)
model = PeftModel.from_pretrained(
    model, ADAPTER, torch_dtype=torch.bfloat16, device_map=device)

processor = AutoProcessor.from_pretrained(BASE)
model.eval()
```

Step 2 — preprocess the image like the dataset

`infer_preprocess.py` – official resize + a real caveat

```
from PIL import Image

# The model card resizes to (1024, 512). NOTE: the paper trains at 128x256.
# Match whatever the checkpoint expects; for this adapter, follow the card.
img = (Image.open("modi_strip.jpg")
        .convert("RGB")
        .resize((1024, 512)))

# Best results need a CLEAN strip: grayscale-ish, denoised, deskewed,
# ~3-4 lines per image. The §2 pipeline produces exactly this shape.
```

! Garbage in → garbage out. The teacher was trained on **denoised, deskewed strips**, not raw phone photos. Feed it the §2 output, not a full page.

Step 3 — build the multimodal prompt (official)

`infer_prompt.py` – official (note: keep the card's exact instruction string)

```
messages = [
    {"role": "user", "content": [
        {"type": "image", "image": img},
        {"type": "text",
         "text": "Transliterate the following Modi script to Devnagri script."},
    ]},
    {"role": "assistant", "content": [{"type": "text"}]},
]

inputs = processor.apply_chat_template(
    messages, add_generation_prompt=True, tokenize=True,
    return_dict=True, return_tensors="pt"
).to(model.device, dtype=torch.bfloat16)

input_len = inputs["input_ids"].shape[-1]
```


Step 4 — the model card's manual decode loop

`infer_loop.py` — official (multinomial sampling = non-deterministic output)

```
import torch.nn.functional as F

ids, attn = inputs["input_ids"], inputs["attention_mask"]
pix = inputs["pixel_values"].to(dtype=model.dtype, device=model.device)

with torch.no_grad():
    while True:
        # token-by-token sampling
        out = model(input_ids=ids, attention_mask=attn, pixel_values=pix)
        probs = F.softmax(out.logits[:, -1, :], dim=-1)
        nxt = torch.multinomial(probs, num_samples=1)
        ids = torch.cat([ids, nxt], dim=-1)
        attn = torch.cat([attn, torch.ones_like(nxt)], dim=-1)
        if nxt.item() == processor.tokenizer.eos_token_id or ids.shape[1] >= 350:
            break

print(processor.decode(ids[0, input_len:], skip_special_tokens=True))
```

Step 4b — a cleaner, faster generate() (reconstructed)

`infer_generate.py` — reconstructed (functionally equivalent, much faster)

```
# Prefer the library's optimized generate(): KV-cache, batching, less code.
with torch.no_grad():
    gen = model.generate(
        **inputs,
        max_new_tokens=350,
        do_sample=False,          # greedy -> deterministic, reproducible
        num_beams=1,             # bump to 3-5 for a quality/latency trade
        repetition_penalty=1.1,  # discourage looping on cursive runs
    )

text = processor.decode(gen[0], input_len:], skip_special_tokens=True)
print(text)
```

✓ Use **do_sample=False** when you need the *same* output every run (eval, regression tests). The card's loop samples, so it varies.

The gotchas that will actually bite you

SYMPTOM	FIX
403 / gated repo	Accept the license on the model page + huggingface-cli login
CUDA OOM on 12B	Use bf16; try device_map="auto", 4-bit (bitsandbytes), or a smaller base
dtype mismatch	Cast pixel_values to model.dtype (the loop does this)
Gibberish output	Wrong image shape/quality — run §2 preprocessing first
Adapter won't load	Base must be google/gemma-3-12b-it, transformers ≥ 4.50
Slow, one image at a time	Batch the processor inputs; use generate() with a KV-cache

Batch many strips for throughput (reconstructed)

`infer_batch.py` – reconstructed (process a whole page's strips at once)

```
def transliterate_batch(strips, max_new=350):
    msgs = [[{"role": "user", "content": [
        {"type": "image", "image": im.convert("RGB").resize((1024, 512))},
        {"type": "text", "text": INSTRUCTION}]]] for im in strips]
    batch = processor.apply_chat_template(
        msgs, add_generation_prompt=True, tokenize=True,
        return_dict=True, return_tensors="pt", padding=True
    ).to(model.device, dtype=torch.bfloat16)
    with torch.no_grad():
        gen = model.generate(**batch, max_new_tokens=max_new, do_sample=False)
    n = batch["input_ids"].shape[-1]
    return [processor.decode(g[n:], skip_special_tokens=True) for g in gen]
```

02

Rebuild the data

Preprocess real docs, generate synthetic twins, and check for leakage.

Load MoDeTrans & SynthMoDe (official schema)

load_data.py – official (the schemas you saw in Part 1)

```
from datasets import load_dataset

# real: filename | image (PIL) | text (Devanagari)
real = load_dataset("historyHulk/MoDeTrans", split="train")
# synthetic: filename | image1 | image2 | text (text == real text)
syn = load_dataset("historyHulk/SynthMoDe", split="train")

print(real[0]["filename"], "|", real[0]["text"][:40])
print(real)  # 2,043 rows ; syn also 2,043 rows
# each real["image"] is a denoised Modi strip; the label is real["text"]
```

i **image1** = MarathiCursive render, **image2** = Noto-Sans-Modi render. Same text as the matching real row.

Real-doc preprocessing — the pipeline (reconstructed)

preprocess.py — reconstructed (mirrors the paper's described steps)

```
import cv2, numpy as np

def preprocess_modi(bgr):
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)          # 1) grayscale
    den = cv2.bilateralFilter(gray, d=3, sigmaColor=75,    # 2) edge-preserving
                                sigmaSpace=75)           #   denoise (k=3)
    th = gaussian_adaptive_threshold(den)                 # 3) binarize (next slide)
    desk = deskew(th)                                     # 4) straighten
    return desk

# paper's order: split -> grayscale -> denoise -> (manual fix) -> threshold -> deskew
# "manual fix" = human + Photoshop for the worst pages (no code can fully replace it)
```

Gaussian adaptive threshold $t = k \cdot \sigma$ (reconstructed)

threshold.py – reconstructed (the appendix formula, in OpenCV)

```
def gaussian_adaptive_threshold(gray, blur=5, block=31, k=2):  
    # paper:  $t = k * \sigma$ ,  $0.5 < k < 2$ , applied after a Gaussian blur  
    blurred = cv2.GaussianBlur(gray, (blur, blur), 0)  
    # OpenCV's adaptive threshold implements the same local-Gaussian idea:  
    return cv2.adaptiveThreshold(  
        blurred, 255,  
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
        cv2.THRESH_BINARY,  
        blockSize=block,      # local neighborhood (odd)  
        C=2)                  # subtracted constant (acts like the offset)  
# Pixels brighter than the local threshold -> white(255), else -> black(0).
```


Deskew — straighten the shirokekha (reconstructed)

deskew.py – reconstructed (the horizontal shirokekha makes this reliable)

```
def deskew(binary):  
    # find foreground pixels, fit a minimum-area rectangle, rotate to level  
    coords = np.column_stack(np.where(binary < 255))  
    angle = cv2.minAreaRect(coords)[-1]  
    angle = -(90 + angle) if angle < -45 else -angle  
    (h, w) = binary.shape  
    M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)  
    return cv2.warpAffine(binary, M, (w, h),  
                          flags=cv2.INTER_CUBIC,  
                          borderMode=cv2.BORDER_REPLICATE)
```

i Deskew matters more for Modi than most scripts: the **shirokekha** top-line gives a strong horizontal reference to align to.

Segment a page into 3-4 line strips (reconstructed)

segment.py – reconstructed (projection-profile line cutting)

```
def segment_lines(binary, n_strips=4):  
    # horizontal projection: count black pixels per row, cut at the gaps  
    proj = (binary < 128).sum(axis=1)  
    valleys = find_low_density_rows(proj)          # whitespace between lines  
    bands = group_into_strips(valleys, n_strips)  # ~3-4 lines, ~40 words each  
    return [binary[y0:y1, :] for (y0, y1) in bands]  
  
# Why strips? A full page is too long for the context window and hurts accuracy.  
# The dataset itself is built from strips of ~3-4 lines (Part 1, preprocessing).
```

The synthetic engine — text → Modi image (reconstructed)

synth_render.py – reconstructed (the core text->image step)

```
from PIL import Image, ImageDraw, ImageFont

# real font files (installed via: apt-get install fonts-noto-core)
NOTO_MODI = "/usr/share/fonts/truetype/noto/NotoSansModi-Regular.ttf"
# MarathiCursive: github.com/MihailJP/MarathiCursive (download the .ttf)
CURSIVE = "fonts/MarathiCursive.ttf"

def render_modi(text, font_path, size=(1024, 512), fs=48):
    canvas = Image.new("RGB", size, "white")
    d = ImageDraw.Draw(canvas)
    font = ImageFont.truetype(font_path, fs)
    d.multiline_text((40, 40), text, fill="black", font=font, spacing=18)
    return canvas
```



⚠ Subtlety: Modi fonts expect text in **Modi Unicode**. The authors map the Devanagari labels into Modi glyphs before rendering. A naive render of Devanagari with a Modi font won't be faithful — a transliteration map is required.

Two images per row — the SynthMoDe trick (reconstructed)

build_synth.py — reconstructed (each text -> 2 font renders, label copied)

```
import os, json

def build_synthmode(real_dataset, out_dir):
    rows = []
    for ex in real_dataset:
        text, fn = ex["text"], ex["filename"]
        img1 = render_modi(text, CURSIVE)
        img2 = render_modi(text, NOTO_MODI)
        img1.save(f"{out_dir}/img1/{fn}")
        img2.save(f"{out_dir}/img2/{fn}")
        rows.append({"filename": fn, "text": text})
    json.dump(rows, open(f"{out_dir}/labels.json", "w"), ensure_ascii=False)
```

Make synthetic harder — Phase-1 augmentation (illustrative)

augment.py – illustrative (the paper's synthetic was clean; this is the upgrade)

```
def degrade(img):                # push synthetic toward real-world ugliness
    a = np.array(img)
    a = add_gaussian_noise(a, sigma=12)      # sensor / scan grain
    a = random_ink_bleed(a, p=0.3)           # cursive ink spread
    a = paper_texture(a, alpha=0.15)         # aged-paper background
    a = random_rotate(a, max_deg=3)         # slight skew
    a = jpeg_recompress(a, q=random.randint(40,75)) # scan artifacts
    return Image.fromarray(a)
# Domain randomization closes the synthetic->real gap BEFORE you spend
# scarce real labels. This is the heart of a "high-fidelity synthetic engine".
```

⚠ Leakage check — do the two sets share data?

leakage_check.py – illustrative (run this on ANY 'augmented' dataset)

```
real_text = {r["filename"]: r["text"] for r in real}
syn_text  = {s["filename"]: s["text"] for s in syn}

shared_files = set(real_text) & set(syn_text)
same_label   = sum(real_text[f] == syn_text[f] for f in shared_files)

print(len(shared_files), "shared filenames")          # -> 2043
print(same_label, "identical text labels")            # -> ~2043 (by construction)
# CONCLUSION: images differ (real scan vs font render), but the TEXT TARGETS
# are the same set. "Train on real, test on synth" => unseen images, SEEN text.
```

⚠ This is the single most important diagnostic in the deck. The overlap is at the **label** level, not the pixel level — state it that precisely.

Build a leakage-safe split (illustrative)

safe_split.py – illustrative (hash the content to prevent text leakage)

```
import hashlib

def split_by_text(rows, ratios=(0.8, 0.1, 0.1)):
    # hash the LABEL, not the filename -> identical sentences land together
    def bucket(text):
        h = int(hashlib.md5(text.encode("utf-8")).hexdigest(), 16) % 100
        return "train" if h < 80 else "val" if h < 90 else "test"
    out = {"train": [], "val": [], "test": []}
    for r in rows:
        out[bucket(r["text"])].append(r)
    return out
# Guarantees a test sentence never appeared (in any rendering) during training.
```

03

Train teacher + student

LoRA fine-tune the teacher, then distill into a tiny student.

The two-stage plan (reconstructed)

STAGE A — TEACHER

- ▶ Load a VLM (Gemma-3-12B-it / LLaMA-3).
- ▶ Attach **LoRA** (rank 64), freeze the base.
- ▶ Train on image→Devanagari with **CE loss**.

STAGE B — STUDENT

- ▶ Small custom decoder (parallel-attn + QK-norm).
- ▶ Teacher frozen; student mimics it.
- ▶ Loss = **CE + L2 + KL** (knowledge distillation).

i Config recap: AdamW, cosine schedule, lr 1e-6 (teacher) / 1e-4 (student), batch 4 × grad-accum 8, BF16.

Attach LoRA to the VLM (reconstructed)

teacher_lora.py – reconstructed (rank 64 is the paper's value)

```
from transformers import AutoModelForImageTextToText
from peft import LoraConfig, get_peft_model

model = AutoModelForImageTextToText.from_pretrained(
    "google/gemma-3-12b-it", torch_dtype=torch.bfloat16, device_map="auto")

lora = LoraConfig(
    r=64, lora_alpha=128, lora_dropout=0.05, bias="none",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], # attention proj
    task_type="CAUSAL_LM")

model = get_peft_model(model, lora)
model.print_trainable_parameters() # ~ <1% of params are trainable
```

Forward pass + cross-entropy (reconstructed)

teacher_forward.py – reconstructed (L_CE on the LoRA params only)

```
import torch.nn.functional as F

def teacher_step(model, batch):
    out = model(pixel_values=batch["pixel_values"],
                input_ids=batch["input_ids"],
                attention_mask=batch["attention_mask"])
    logits = out.logits[:, :-1, :]          # predict next token
    labels = batch["input_ids"][:, 1:]      # shifted targets
    ce = F.cross_entropy(
        logits.reshape(-1, logits.size(-1)),
        labels.reshape(-1),
        ignore_index=PAD_ID)               # mask padding
    return ce, out.hidden_states           # hidden_states feed the student
```

The teacher training loop (reconstructed)

teacher_train.py – reconstructed (AdamW + cosine, accum 8)

```
from torch.optim import AdamW
from transformers import get_cosine_schedule_with_warmup

opt = AdamW(model.parameters(), lr=1e-6, weight_decay=0.01)
sched = get_cosine_schedule_with_warmup(opt, num_warmup_steps=200,
                                       num_training_steps=TOTAL_STEPS)

ACCUM = 8
for step, batch in enumerate(loader):
    ce, _ = teacher_step(model, batch)
    (ce / ACCUM).backward()           # gradient accumulation
    if (step + 1) % ACCUM == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step(); sched.step(); opt.zero_grad()
model.save_pretrained("modi-teacher-lora") # saves only the adapter
```

One decoder block: parallel attention + QK-norm (reconstructed)

student_block.py – reconstructed (the paper's two named innovations)

```
import torch, torch.nn as nn

class ParallelBlock(nn.Module):
    """Attention and MLP run in PARALLEL on the same normed input."""
    def __init__(self, d, heads):
        super().__init__()
        self.norm = nn.LayerNorm(d)
        self.q, self.k, self.v = (nn.Linear(d, d) for _ in range(3))
        self.qn, self.kn = nn.LayerNorm(d // heads), nn.LayerNorm(d // heads) # QK-Norm
        self.o = nn.Linear(d, d)
        self.mlp = nn.Sequential(nn.Linear(d, 4*d), nn.GELU(), nn.Linear(4*d, d))
        self.heads = heads
    def forward(self, x):
        h = self.norm(x) # ONE norm feeds both paths
        a = causal_attention(self.qn, self.kn, self.q(h), self.k(h),
                             self.v(h), self.heads)
        return x + self.o(a) + self.mlp(h) # parallel sum
```

Stack N blocks → the MoScNet sizes (reconstructed)

student_model.py – reconstructed (XL = the best 429M config)

```
class MoScNetStudent(nn.Module):
    def __init__(self, d, n_blocks, heads, vocab):
        super().__init__()
        self.embed = nn.Embedding(vocab, d)
        self.blocks = nn.ModuleList(ParallelBlock(d, heads) for _ in range(n_blocks))
        self.head = nn.Linear(d, vocab)
    def forward(self, z):
        # z = concat(image_embeds, text_embeds)
        for blk in self.blocks: z = blk(z)
        return self.head(z)

# paper variants: S(256,6)=44M M(512,13)=92M L(768,18)=234M XL(1024,21)=429M
student = MoScNetStudent(d=1024, n_blocks=21, heads=16, vocab=VOCAB) # XL
```

The KD loss: CE + L2 + KL (reconstructed)

kd_loss.py – reconstructed ($L = L_{CE} + L_{L2} + L_{KL}$, the paper's Eq. 4)

```
def kd_loss(student_logits, student_hidden,
            teacher_logits, teacher_hidden, labels, T=2.0):
    # 1) primary task: student must get the text right
    ce = F.cross_entropy(student_logits.reshape(-1, V),
                        labels.reshape(-1), ignore_index=PAD_ID)
    # 2) feature mimicry: pull hidden states together (Euclidean / L2)
    l2 = F.mse_loss(student_hidden, teacher_hidden)
    # 3) distribution mimicry: match the teacher's softened outputs (KL)
    kl = F.kl_div(F.log_softmax(student_logits / T, -1),
                 F.softmax(teacher_logits / T, -1),
                 reduction="batchmean") * (T * T)
    return ce + l2 + kl          # ablation: each term adds ~+2-3 BLEU
```

The distillation loop — teacher frozen (reconstructed)

distill_train.py – reconstructed (the core teacher->student transfer)

```
teacher.eval(); [p.requires_grad_(False) for p in teacher.parameters()]
opt = AdamW(student.parameters(), lr=1e-4)      # student lr is higher

for batch in loader:
    with torch.no_grad():                       # teacher gives targets
        t_logits, t_hidden = teacher_forward(teacher, batch)
    s_logits, s_hidden = student(batch["z"])     # student learns
    loss = kd_loss(s_logits, s_hidden, t_logits, t_hidden, batch["labels"])
    (loss / ACCUM).backward()
    if step % ACCUM == 0:
        opt.step(); sched.step(); opt.zero_grad()
# Ship ONLY the student: 429M params, ~163x smaller than a 70B teacher.
```

- ✓ At deploy time the teacher is discarded. This is why distillation is worth the trouble: a cheap, fast model that punches above its size.

Scale it across GPUs with Accelerate (reconstructed)

`accelerate_train.py` – reconstructed (one wrapper, many GPUs)

```
# accelerate handles DDP/FSDP, mixed precision, grad-accum for you
from accelerate import Accelerator
acc = Accelerator(mixed_precision="bf16", gradient_accumulation_steps=8)

student, teacher, opt, loader = acc.prepare(student, teacher, opt, loader)
for batch in loader:
    with acc.accumulate(student):
        loss = kd_loss(*forward_both(student, teacher, batch))
        acc.backward(loss)
        opt.step(); opt.zero_grad()
# Launch: accelerate launch --multi_gpu --num_processes 8 distill_train.py
# Paper used 16 nodes x 8 H100 = 128 GPUs, BF16.
```

04

Evaluate & interrogate

BLEU, CER, and the five questions a reviewer must force.

BLEU — the paper's headline metric (illustrative)

eval_bleu.py – illustrative (reproduces the paper's metric)

```
import sacrebleu

def corpus_bleu(preds, refs):
    # sacrebleu expects refs as a list of reference-lists
    return sacrebleu.corpus_bleu(preds, [refs]).score

preds = [transliterate(im) for im in test_images]
score = corpus_bleu(preds, test_texts)
print(f"BLEU: {score:.2f}")      # paper: MoScNet-XL = 51.0 on MoDeTrans

# BLEU rewards n-gram overlap. Good for ranking systems; BLIND to WHICH
# characters are wrong -> always report CER alongside it.
```

CER / WER — the metric the paper omits (illustrative)

eval_cer.py – illustrative (answers 'how wrong, character by character?')

```
import jiwer

def cer(pred, ref):
    return jiwer.cer(ref, pred)      # Character Error Rate (lower is better)
                                     # (substitutions+insertions+deletions)/chars

def wer(pred, ref):
    return jiwer.wer(ref, pred)

cers = [cer(p, r) for p, r in zip(preds, test_texts)]
print(f"mean CER: {sum(cers)/len(cers):.3f}")
# CER exposes the bha/ma, ka/pha, missing-anusvara errors the paper LISTS
# but never quantifies. For a transliteration tool, CER is the honest number.
```



If you adopt this model, **CER is your number**, not BLEU — it maps directly to how much an expert must fix.

CER on damaged manuscripts (illustrative)

eval_robustness.py – illustrative (the missing damage benchmark)

```
# The paper reports NO damage-stratified results. Build the test yourself:
def damage(img, mode):
    if mode == "water": return blur_blotch(img, n=6, radius=40)
    if mode == "burn": return darken_edges(img, char=True) # charred borders
    if mode == "fade": return reduce_contrast(img, factor=0.4)
    if mode == "tear": return mask_random_strips(img, k=3)

for mode in ["clean", "water", "burn", "fade", "tear"]:
    imgs = [damage(im, mode) for im in test_images]
    preds = [transliterate(im) for im in imgs]
    print(mode, "CER:", mean(cer(p,r) for p,r in zip(preds, test_texts)))
# Expect CER to climb sharply on burn/water -> quantify the cliff.
```

Does it generalize to unseen scripts e.g. Brahmi? (illustrative)

`eval_zeroshot.py` – illustrative (sets correct expectations)

```
# The paper's "zero-shot" = same TEXT, synthetic images. It NEVER tests a
# different script. To actually test cross-script transfer:
brahmi_imgs, brahmi_refs = load_script_dataset("brahmi") # you must source this
preds = [transliterate(im) for im in brahmi_imgs]
print("Brahmi CER:", mean(cer(p, r) for p, r in zip(preds, brahmi_refs)))

# HONEST EXPECTATION: near-total failure. The adapter learned Modi glyph ->
# Devanagari mappings; Brahmi is a different glyph system. Zero-shot transfer
# across scripts is NOT a claim this model makes or supports.
```

 Reviewer takeaway: never let a vague 'zero-shot' label imply more generalization than was tested. Here it means *new images, seen text* — nothing about new scripts.

Validate the ground-truth labels (illustrative)

eval_agreement.py – illustrative (quantifies label trustworthiness)

```
# The paper used a small expert pool + cross-verification, but reports NO
# inter-annotator agreement. Measure it: have 2-3 experts label the SAME 100 docs.
import itertools
def pairwise_disagreement(label_sets):      # label_sets: {expert: {fn: text}}
    fns = set.intersection(*[set(d) for d in label_sets.values()])
    experts = list(label_sets)
    for a, b in itertools.combinations(experts, 2):
        d = mean(cer(label_sets[a][f], label_sets[b][f]) for f in fns)
        print(f"{a} vs {b}: char-level disagreement = {d:.3f}")
# High disagreement => the 'ground truth' itself is uncertain (expert bias).
```

Does it memorize sensitive land-record data? (illustrative)

eval_privacy.py – illustrative (membership/extraction-style probe)

```
# The paper asserts no sensitive data; records are centuries old. Still PROBE it:
def extraction_test(model, train_prefixes):
    leaks = 0
    for prefix in train_prefixes:          # feed start of a TRAIN sample
        out = greedy_continue(model, prefix, n=80)
        if out in TRAIN_TEXT_SET and out not in VAL_TEXT_SET:
            leaks += 1                     # verbatim training-text recall
    return leaks / len(train_prefixes)

print("verbatim-recall rate:", extraction_test(model, sampled_train_prefixes))
# Also: insert canary strings into training data and test recall after training.
```

! For old administrative records the privacy risk is low, but the **method** is exactly what you'd run on modern health/finance data — memorization is a deployment blocker there.

One consolidated report (illustrative)

eval_report.py – illustrative (the scorecard the paper should ship)

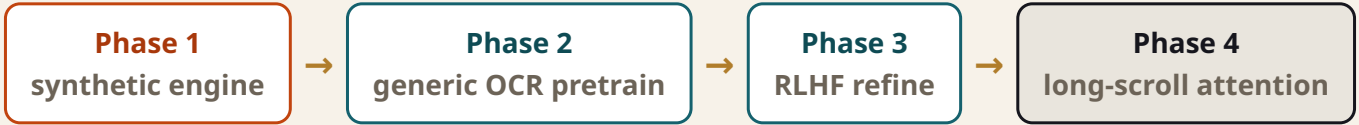
```
def evaluation_report(model, test):
    return {
        "BLEU": corpus_bleu(preds(model, test), test.texts),
        "CER": mean(cer(p,r) for p,r in pairs(model, test)),
        "CER_by_era": {e: cer_subset(model, test, era=e) for e in ERAS},
        "CER_damage": {m: cer_damage(model, test, m) for m in DAMAGE_MODES},
        "leakage": text_overlap(train.texts, test.texts), # must be 0
        "verbatim": extraction_test(model, train.prefixes),
    }
# A model is only as trustworthy as the questions you forced it to answer.
```

05

Build it from scratch

A 4-phase pipeline if you started this project today.

The four phases at a glance



PHASE	PURPOSE
1 · High-fidelity synthetic	More fonts + realistic noise → cheap, large pretraining data
2 · Pretrain on generic OCR	Teach 'read images→text' before Modi-specific tuning
3 · RLHF / DPO	Fix nuances synthetic data can't teach, using human preferences
4 · Sliding-window attention	Read long scrolls that exceed the context window

PHASE 1

A high-fidelity synthetic engine (illustrative)

phase1_engine.py – illustrative (diversity + realism = transferable)

```
FONT_Z00 = glob("fonts/modi/*.ttf")          # collect MANY Modi fonts, not 2

def synth_sample(text):
    font = random.choice(FONT_Z00)           # font diversity
    img = render_modi(text, font, fs=random.randint(36, 60))
    img = degrade(img)                       # noise/bleed/texture (slide 18)
    img = random_background(img)             # palm-leaf, paper, parchment
    return img, text

# Generate 100k-1M pairs cheaply. The closer synthetic looks to real decay,
# the less scarce real data you need later. This is the project's foundation.
```

Domain randomization knobs (illustrative)

phase1_randomize.py – illustrative (the parameters that close the gap)

```
RANDOMIZE = dict(
    font          = FONT_Z00,                # many handwriting styles
    font_size     = (32, 64),
    rotation      = (-4, 4),                 # mild skew
    ink_color      = [(20,20,20), (60,40,30)], # faded blacks/browns
    noise_std     = (5, 18),
    blur          = (0, 1.5),
    bg_texture     = ["paper", "parchment", "palmleaf"],
    jpeg_q        = (35, 80),
)
# Train across the FULL range so the model can't overfit to one clean look.
# Rule of thumb: if a human finds your synthetic 'too perfect', so will the gap.
```

PHASE 2

Pretrain the VLM on generic OCR (illustrative)

```
phase2_pretrain.py – illustrative (curriculum: generic OCR first)
```

```
# Before Modi-specific tuning, teach the base model the GENERIC skill of  
# turning text-images into strings (cheap, abundant data).  
ocr_data = load_dataset("mixed", sources=["mjsynth", "synthtext", "iam"])  
  
def ocr_pretrain(vlm, ocr_data, steps=200_000):  
    for batch in ocr_data: # image -> text, any script  
        loss = vlm(**batch).loss  
        loss.backward(); step()  
    return vlm  
  
# Result: the vision encoder + projector learn "read pixels as characters".  
# THEN attach LoRA and fine-tune on Modi -> faster convergence, higher ceiling.
```

i This is why the paper's model also scores well on ICDAR scene-text: the OCR skill is a shared substrate. Build that substrate first.

Curriculum: generic → Modi (illustrative)

phase2_curriculum.py – illustrative (spend real labels last)

```
stages = [  
    ("generic_ocr", ocr_data, {"lr": 1e-4, "steps": 200_000}),  
    ("synth_modi", synth_modi_data, {"lr": 5e-5, "steps": 80_000}),  
    ("real_modi", modetrans_data, {"lr": 1e-5, "steps": 20_000}), # scarce!  
]  
for name, data, cfg in stages:  
    print("=> stage:", name)  
    train(model, data, **cfg) # easy -> hard, abundant -> scarce  
# Save the scarce, expensive REAL data for last, at the lowest learning rate.
```

Why RLHF/DPO — what synthetic data misses (illustrative)

- ▶ Synthetic data teaches **glyph shapes**, not scribal **judgment**: ambiguous ligatures, era-specific spellings, dropped diacritics.
- ▶ Experts can **rank** two candidate transliterations even when they can't write a rule.
- ▶ **DPO** (Direct Preference Optimization) learns from those rankings — simpler & more stable than full PPO-RLHF.

i Collect pairs: model emits A and B for the same strip; an expert picks the better one. That preference signal is the training data.

DPO skeleton with TRL (illustrative)

phase3_dpo.py – illustrative (preference tuning, the pragmatic RLHF)

```
from trl import DPOTrainer, DPOConfig
# preference data: {"prompt": <image+instruction>, "chosen": good, "rejected": bad}
pref = load_dataset("json", data_files="expert_preferences.jsonl")["train"]

trainer = DPOTrainer(
    model=student,           # the model to refine
    ref_model=ref_student,   # frozen reference (anchor)
    args=DPOConfig(beta=0.1, learning_rate=5e-6, per_device_train_batch_size=2),
    train_dataset=pref,
    processing_class=processor)
trainer.train()
# beta controls how far you may drift from the reference. Start small.
```

Sliding-window attention for long scrolls (illustrative)

phase4_window.py – illustrative (local attention = linear in length)

```
# A 2-metre scroll won't fit in the context window. Two complementary moves:  
# (A) MODEL-LEVEL: limit each token to a local window of W previous tokens.  
def sliding_window_mask(seq_len, W=512):  
    i = torch.arange(seq_len)[: , None]  
    j = torch.arange(seq_len)[None, :]  
    return (j <= i) & (j > i - W)    # causal AND within window W  
  
# attn_scores = attn_scores.masked_fill(~mask, float("-inf"))  
# Cost drops from  $O(n^2)$  to  $O(n*W)$ ; the model reads arbitrarily long sequences.
```

Inference-time tiling + stitch (illustrative)

phase4_tiling.py – illustrative (works even without model changes)

```
def transliterate_long(scroll_img, win=1024, overlap=128):
    tiles = sliding_tiles(scroll_img, width=win, stride=win - overlap)
    texts = [transliterate(t) for t in tiles]          # per-tile decode
    return stitch(texts, overlap_chars=estimate_overlap(overlap))

def stitch(chunks, overlap_chars):
    out = chunks[0]
    for c in chunks[1:]:
        k = best_overlap(out[-overlap_chars:], c[:overlap_chars]) # align seam
        out += c[k:]                                                # de-duplicate
    return out
# Overlap + seam-merge prevents lost or doubled characters at tile boundaries.
```

The repository you'd end up with

```
modi-translit/
├── data/
│   ├── preprocess.py      # grayscale, denoise, threshold, deskew, segment
│   ├── synth_engine.py    # font-zoo render + degrade (Phase 1)
│   └── leakage_check.py    # text-level overlap + safe split
├── train/
│   ├── teacher_lora.py    # PEFT fine-tune
│   ├── student.py         # parallel-attn + QK-norm decoder
│   ├── distill.py         # CE + L2 + KL loop
│   └── dpo.py             # Phase 3 preference tuning
├── eval/
│   ├── metrics.py        # BLEU + CER/WER
│   └── probes.py         # robustness, zero-shot, privacy
├── serve/ api.py         # FastAPI + human-review hooks
└── infer.py              # load base+adapter, tile long scrolls
```

06

Ship & close

Serving, human-in-the-loop, testing, and the takeaways.

A minimal inference API (reconstructed)

api.py – reconstructed (wrap the model behind one endpoint)

```
from fastapi import FastAPI, UploadFile
from PIL import Image
import io

app = FastAPI()

@app.post("/transliterate")
async def transliterate_ep(file: UploadFile):
    raw = Image.open(io.BytesIO(await file.read())).convert("RGB")
    strip = preprocess_modi_pil(raw)           # reuse the $2 pipeline
    text = transliterate(strip)                # greedy, deterministic
    return {"devanagari": text, "needs_review": True}  # never auto-trust

# Run: uvicorn api:app --host 0.0.0.0 --port 8000
```

Human-in-the-loop is the product, not a feature



- ▶ Surface the model's output as a **draft**, with low-confidence regions highlighted.
 - ▶ Every expert correction becomes new training data (and DPO preference pairs).
 - ▶ This loop is exactly how the original MoDeTrans labels were made — keep it.
- ✓ For an assistive system in history (or health/finance), the human is the safety boundary. Design the UI around correction, not blind acceptance.

Don't overpromise — set the contract

- ▶ **State the metric honestly:** report CER, not just BLEU; report it *by era and by damage level*.
- ▶ **Scope the claim:** 'assistive transliteration for these three eras', not 'reads any Modi document'.
- ▶ **Flag uncertainty** in the output; never present a guess as authoritative.
- ▶ **Keep provenance:** store which model version + preprocessing produced each result.
- ▶ **Privacy by default:** probe for memorization before touching any modern, sensitive corpus.

Cost & latency you should size up front

COMPONENT	PROFILE	NOTE
Teacher (12B/70B)	High VRAM, slow	Training / distillation only — not for serving
Student (429M)	Low VRAM, fast	The thing you actually deploy
Per-strip latency	Dominated by tokens generated	Cap max_new_tokens; greedy decode
Long scroll	× number of tiles	Tile + stitch; parallelize tiles
Carbon	≈40 kg CO ₂ / train-hour	Report it; distillation slashes run-time cost

A regression harness so quality can't silently drop (reconstructed)

tests/test_quality.py – reconstructed (CI gate on model quality)

```
GOLDEN = load_json("tests/golden_50.json")    # 50 hand-checked strips

def test_no_cer_regression():
    cers = [cer(transliterate(load(g["img"]))), g["text"]] for g in GOLDEN
    mean_cer = sum(cers) / len(cers)
    assert mean_cer <= BASELINE_CER + 0.01, f"CER regressed: {mean_cer:.3f}"

def test_deterministic():
    a = transliterate(load(GOLDEN[0]["img"]))    # do_sample=False
    b = transliterate(load(GOLDEN[0]["img"]))
    assert a == b, "non-deterministic decode in eval path"
```

- ✓ Treat model quality like code: a golden set + a CER ceiling in CI catches the day a 'small change' quietly breaks transliteration.

The checklist before you call it done

- ▶ Pin versions: torch, transformers, peft.
- ▶ Seed everything; log **do_sample** & decode params.
- ▶ Record exact image size & preprocessing.
- ▶ Snapshot dataset split hashes.
- ▶ Save the adapter + base commit hash.
- ▶ Log BLEU **and** CER, by era + damage.
- ▶ Store the leakage-check result (0 overlap).
- ▶ Document compute & CO₂.

RECAP

The code you can now write

- ▶ **Run** the released teacher end-to-end.
- ▶ **Preprocess** real Modi docs (OpenCV).
- ▶ **Generate** synthetic data with font renders.
- ▶ **Detect** label-level leakage.
- ▶ **LoRA-tune** a VLM; **distill** a tiny student.
- ▶ **Measure** BLEU, CER, robustness, privacy.
- ▶ **Stage** a 4-phase from-scratch build.
- ▶ **Serve** it with HITL + CI quality gates.



You came in able to read about the model. You leave able to rebuild, evaluate, and ship one.

GLOSSARY

Code/ML terms used in this deck

PEFT/LoRA	Train tiny low-rank add-ons, freeze the base	CER/WER	Character/Word Error Rate (lower better)
KD	Knowledge distillation: student mimics teacher	DPO	Preference tuning without a reward model
CE / KL / L2	The three distillation loss terms	Domain randomization	Vary synthetic data so real looks in-distribution
QK-Norm	Normalize attention queries+keys for stability	Sliding window	Local attention → linear-cost long sequences
Parallel block	Attention + MLP computed side-by-side	HITL	Human-in-the-loop review & correction

Primary materials & tools

- ▶ **Paper / config / losses:** arXiv 2503.13060 (ICDAR 2025).
- ▶ **Inference code:** model card — huggingface.co/historyHulk/ModiTrans-12B-Gemma-Teacher.
- ▶ **Datasets:** MoDeTrans & SynthMoDe dataset cards (schemas, sizes).
- ▶ **Libraries:** transformers · peft · trl · accelerate · sacrebleu · jiwer · opencv.
- ▶ **Fonts:** Noto Sans Modi · MarathiCursive (MihailJP).

! Reconstructed/illustrative code reproduces methods the paper *describes*; tune hyperparameters and validate before production use.

CLOSE

Where to point your next hour

Start at **§1**: get one image transliterated on a GPU. Then walk upstream — **§2 data**, **§3 training**, **§4 eval** — and you'll understand every number in the paper from the inside.

i Pair this with Part 1 (The WHY) for the full picture: motivation and judgement there, mechanics and code here.